Бенчмарк ресурсов

Английский оригинал: BENCHMARK.md.

Измерено на эталонном профиле нагрузки. Числа ниже получены `scripts/run_benchmark.py`; пересобирайте их, а не правьте вручную.

Проверочное окружение

- ОС: Linux-6.8.0-1055-gke-x86_64-with-glibc2.41
- Ядро: 6.8.0-1055-gke
- ЦП: x86_64 (4 логических ядра)
- ОЗУ: 15990 МиБ
- Python: 3.13.15
- dpkt: 1.9.8

Эталонное окружение в спецификации — Linux x86-64, 4 vCPU, 8 ГБ ОЗУ. Числа ниже получены на машине, показанной выше.

Вход

- Входной захват: 95.66 МиБ
- Прочитано пакетов: 250000
- TCP-сессий: 1000
- Диагностик чтения: 0
- Целевой профиль: 250000 пакетов, 1000 сессий, до 100 МиБ, ответы до 65536 байт

Тайминги

Фаза	Секунды
Разбор (read_capture)	7.29
Сессии (build_sessions)	0.70
Пересборка (reassemble)	1.84
Экспорт (export_capture)	2.27
Итого	12.10

Потоковый режим

`process_streaming` читает захват блоками и пишет каждую сессию, как только она больше не может получать пакеты, поэтому закрытые сессии покидают память. Выход идентичен обычному режиму; различается только профиль ресурсов. Пик потокового режима измеряется в собственном процессе, чтобы они не делили отметку максимума.

Метрика	Обычный	Потоковый
Всего секунд	12.10	11.48
Пик RSS (МиБ)	350	224
Выходной JSON (МиБ)	122.44	122.44

Потоковый прогон сбросил 1000 сессий.

Потоковый режим использовал меньше памяти.

Память и выход

- Пик RSS процесса: 350 МиБ
- Выходной JSON: 122.44 МиБ

Выводы и ограничения

- Пиковая память — 350 МиБ, заметно ниже порога 4 ГиБ.
- Общее время — 12.10 с, заметно ниже порога 5 минут.

Наблюдения:

- Разбор доминирует по времени. Он строит один Python-объект на пакет, и это цена того, что поля заголовка остаются доступны для провенанса.
- Нормализованный JSON (122.44 МиБ) крупнее входа (95.66 МиБ), потому что байты потока хранятся в base64, а каждый наблюдаемый диапазон несёт запись провенанса.
- Память масштабируется с размером захвата: все пакеты держатся в памяти сразу. На этом профиле это приемлемо; для захвата в несколько раз больше тот же подход потребовал бы потокового режима или отображения в память.

Если нужно поддержать более крупный захват, первый шаг — перестать держать все объекты пакетов сразу: разбирать и пересобирать сессия за сессией и писать диапазоны провенанса на диск по мере получения. Это меняет пиковую память на второй проход по файлу.

Развёртка по размерам: обычный против потокового

Один и тот же захват нормализуется дважды, сначала обычным, затем потоковым режимом, поэтому два режима сравниваются на идентичном входе. Каждый размер — отдельный сгенерированный захват с той же формой сессий и сообщений; `packets` масштабируется с размером. Память — пиковый резидентный набор процесса.

Вход	Пакетов	Сессий	Обычный с	Обычный пик МиБ	Потоковый с	Потоковый пик МиБ	Выход МиБ
9.57 МиБ	25000	100	1.10	58	1.06	54	12.20
47.84 МиБ	125000	500	6.10	186	5.71	167	61.12
90.88 МиБ	237500	950	11.46	336	10.86	293	116.32

Разбивка фаз по размерам

Секунды по фазам обычного конвейера. Пересборка включает идентификацию сессий.

Вход	Разбор	Сессии	Пересборка	Экспорт	Итого
9.57 МиБ	0.69	0.067	0.16	0.19	1.10
47.84 МиБ	3.73	0.357	0.96	1.05	6.10
90.88 МиБ	6.83	0.646	1.84	2.13	11.46

Захват с дефектами

`tests/corpus/corpus_capture_defects.pcapng` мал, но несёт ретрансмиссию, сегмент не по порядку, пропуск и конфликтующее перекрытие. Он измерен, чтобы подтвердить, что диагностики не ухудшают пропускную способность на этом размере.

Метрика	Обычный	Потоковый
Вход	1.7 КиБ	тот же
Пакетов	18	18
Сессий	1	1
Секунд	0.021	0.022
Пик RSS (МиБ)	29	29

Выделение памяти Python (tracemalloc, 10 МиБ)

Пиковая память, отслеженная `tracemalloc` во время обычного прогона по захвату 10 МиБ. Это считает только выделения Python, а не память, которую интерпретатор или библиотеки держат вне аллокатора, поэтому значение ниже RSS процесса.

Метрика	Значение
Вход	9.57 МиБ
Пакетов	25000
Пик tracemalloc	31 МиБ
RSS процесса	93 МиБ